QuestDB: Competitive Comparison

vs. kdb+, InfluxDB, TimescaleDB, and ClickHouse

Contents

Introduction . . . 2
QuestDB vs. kdb+ . . . 3
- Programming model: SQL vs. q . . . 3
- AI and LLM readiness . . . 4
- Architecture: open data vs. proprietary . . . 4
- Concurrency and crash recovery . . . 4
- A note on benchmarks . . . 4
- Feature comparison . . . 5
- Summary . . . 7

QuestDB vs. InfluxDB . . . 8
- Ingestion benchmark . . . 8
- Query benchmark: Single-groupby . . . 8
- Query benchmark: Double-groupby . . . 9
- Query benchmark: Heavy analytical . . . 9
- Data model differences . . . 9
- Query language: SQL vs. Flux . . . 10
- QuestDB time-series SQL extensions . . . 10
- Ecosystem and integrations . . . 10
- Performance summary . . . 11

QuestDB vs. TimescaleDB . . . 12
- Ingestion benchmark . . . 12
- Query benchmark: Single-groupby . . . 12
- Query benchmark: Double-groupby . . . 12
- Query benchmark: Heavy analytical . . . 13
- ClickBench: OLAP workload . . . 13
- ASOF JOIN comparison . . . 15
- Architecture . . . 15

QuestDB vs. ClickHouse . . . 16
- Ingestion benchmark . . . 16
- Query benchmark: Single-groupby . . . 16
- Query benchmark: Double-groupby . . . 17
- Query benchmark: Heavy and lastpoint . . . 17
- ClickBench: Web analytics workload . . . 17
- Architecture: CREATE TABLE comparison . . . 19
- QuestDB time-series SQL extensions . . . 19
- Openness and data formats . . . 20

Further reading . . . 21

Introduction

QuestDB is an open source time-series database engineered for low-latency ingestion and analytics. Built from scratch with a zero-GC Java core and focused C++ and Rust components, it stores data in a three-tier architecture (streaming WAL files, local binary columnar storage, and Parquet on object storage) and queries all tiers through a single SQL engine with JIT compilation and SIMD execution.

Organizations evaluating time-series databases face a crowded landscape. This document provides a factual comparison of QuestDB against four alternatives that engineers commonly evaluate: kdb+, InfluxDB, TimescaleDB, and ClickHouse. Each comparison draws from published benchmarks, architectural analysis, and feature-by-feature evaluation.

Benchmarks in this document use two open source suites: the **Time Series Benchmark Suite (TSBS)** for ingestion and time-series queries, and **ClickBench** for OLAP-style analytical queries. Unless otherwise noted, the benchmark environment is an AWS EC2 r8a.8xlarge instance (32 vCPU, 256 GB RAM, AMD EPYC) with GP3 EBS storage (20,000 IOPS, 1 GB/s throughput), running default configurations for all databases tested.

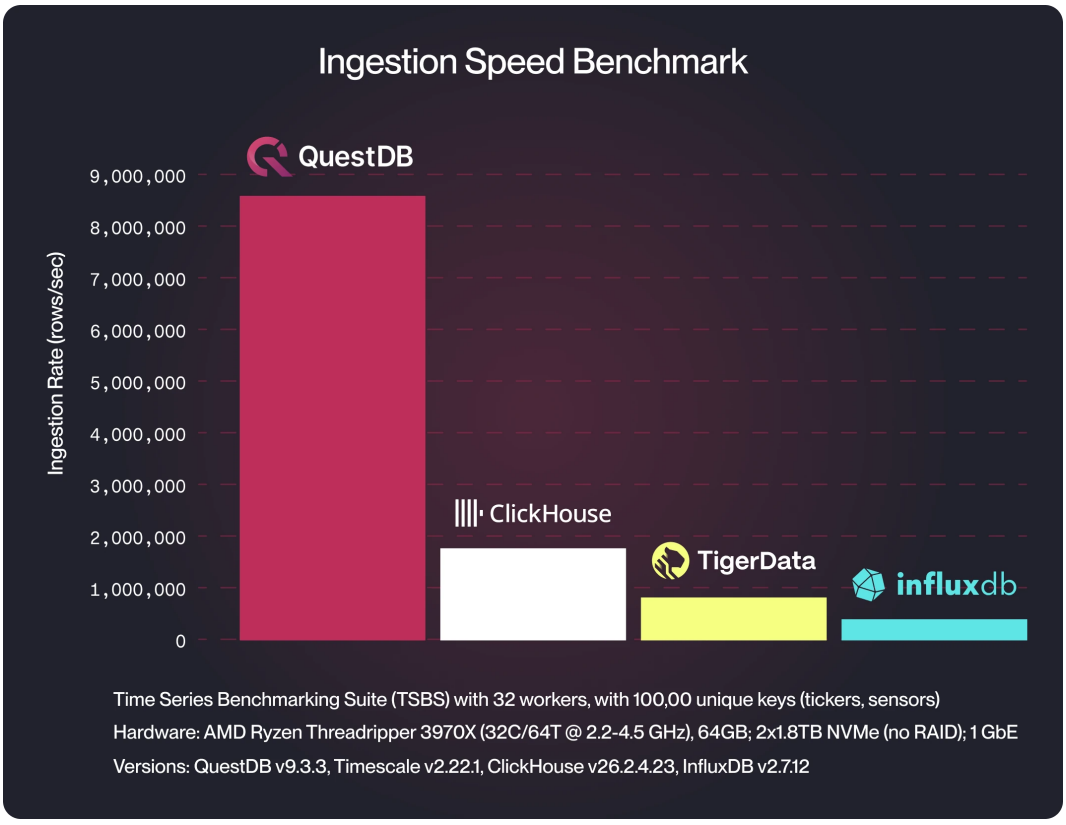

Figure 1: QuestDB benchmark results, Q1 2026

QuestDB consistently delivers over 8 million rows per second ingestion throughput across cardinality levels using the TSBS cpu-only workload with 32 concurrent connections.

	QuestDB	kdb+	InfluxDB	TimescaleDB	ClickHouse
License	Apache 2.0	Proprietary	v1: MIT, v2: Prop.	Apache 2.0 + TSL	Apache 2.0
Query language	SQL	q	InfluxQL/Flux/SQL	SQL (PostgreSQL)	SQL (CH dialect)
Implementation	Java, C++, Rust	k/q	Go	C (PG ext)	C++
Architecture	Three-tier	In-mem + HDB	TSM	PG extension	MergeTree
TS focus	Native	Native	Native	Extension	General OLAP
Ingestion	ILP, PGWire, HTTP	Proprietary	ILP, HTTP	JDBC, ODBC	HTTP, Native, Kafka
Storage	Columnar + Parquet	Proprietary	TSM (prop.)	PG pages	MergeTree (prop.)
Open formats	Parquet, Iceberg	No	No	No	Parquet (I/O)
Mat. views	Built-in, incremental	DIY (q)	Cont. queries	PG mat views	Agg. MergeTree
Replication	Object store	DIY (q)	Enterprise only	PG streaming	CH Keeper
Benchmarks	Open (TSBS, CB)	Restricted	Open	Open	Open (CB)
TSBS 4K hosts	8–9.5M rows/s	N/A	787K rows/s	1.18M rows/s	1.74M rows/s

QuestDB vs. kdb+

kdb+ is a columnar database built around q, a terse, powerful array language descended from APL. It has a large installed base in capital markets, where it powers trading desks, risk systems, and surveillance platforms.

Programming model: SQL vs. q

QuestDB and kdb+ are both compute engines at their core. The fundamental difference is the programming model.

kdb+ gives users q, a language where a few lines can perform complex vector operations that would take pages of code elsewhere. The trade-off is that q is the answer to every question: query routing, authentication, TLS, materialized views, and replication are all q scripts and q gateways written and maintained by the customer. q is powerful for ad-hoc exploration and quantitative prototyping, but building and maintaining production infrastructure in a niche language carries significant operational weight.

QuestDB delivers solutions expressed in SQL. ASOF JOIN for slippage, HORIZON JOIN for markout curves, SAMPLE BY for time-bucketed aggregations, LATEST ON for last-price lookups, and materialized views that refresh incrementally as data arrives. These primitives compose: a slippage calculation is an ASOF JOIN, a markout curve is a HORIZON JOIN, an ECN scorecard is a HORIZON JOIN grouped by venue.

Define an OHLCV bar once and it stays current in real time:

```
CREATE MATERIALIZED VIEW trades_ohlcv_1s AS
  SELECT
      timestamp, symbol,
      first(price) AS open,
      max(price)   AS high,
      min(price)   AS low,
      last(price)  AS close,
      sum(amount)  AS volume
  FROM trades
  SAMPLE BY 1s;
```

In kdb+, equivalent functionality requires manually coding an RDB update loop and end-of-day flush to the HDB.

AI and LLM readiness

QuestDB is built on open standards: REST API, PostgreSQL wire protocol, standard SQL, Parquet, Arrow. Any LLM or AI agent connects natively. A natural-language question can be translated to SQL, executed over REST or PGWire, and the results piped as Parquet or Arrow directly into pandas, Polars, or any ML framework.

kdb+ exposes a proprietary protocol and the q language. LLMs fail on 57% of q coding tasks on the first attempt, and still fail 26% even with 10 retries. The reasons are structural: q evaluates right-to-left, uses extreme operator overloading, and has almost no public training data compared to SQL.

Architecture: open data vs. proprietary

QuestDB implements a three-tier storage architecture with open formats. Partitions convert to Apache Parquet and move to object storage (S3, Azure Blob, GCS) while remaining fully queryable. QuestDB exports table metadata to Apache Iceberg, Delta Lake, and Apache Hive catalog formats, so data is accessible to any tool in the modern data ecosystem without going through QuestDB.

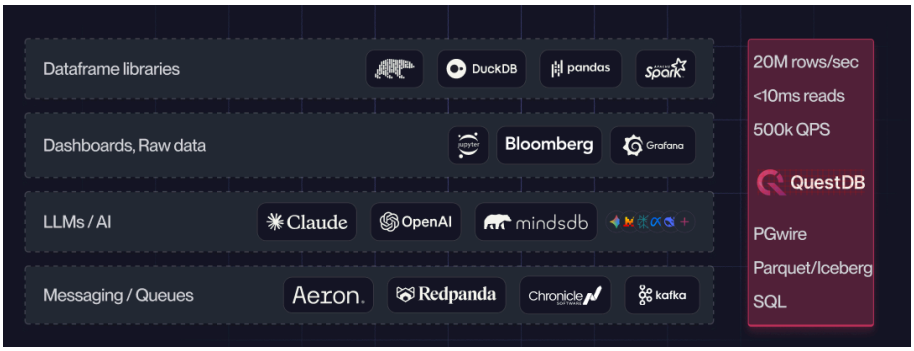

Figure 2: QuestDB data lake interoperability

kdb+ uses a proprietary binary format with coupled storage and compute. Data duplication across RDB (real-time) and HDB (historical) tiers requires careful management, and replication requires custom q scripts.

Concurrency and crash recovery

QuestDB supports multi-threaded reads and writes. Readers never block; concurrent analytical queries execute in parallel while ingestion continues. kdb+'s core event loop is single-threaded, and only one process may write to a given table at a time.

After a crash, QuestDB replays the last few seconds of write-ahead log. kdb+'s RDB must replay the tickerplant log to reconstruct state, which can take minutes.

A note on benchmarks

QuestDB publishes reproducible benchmarks using TSBS and ClickBench. kdb+'s license includes a DeWitt clause that prohibits publishing benchmark results without KX's approval, so independent head-to-head comparisons are not possible. The table below shows QuestDB's standalone TSBS query results.

Environment: AWS r8a.8xlarge (32 vCPU, 256 GiB RAM), EBS GP3 (20k IOPS, 1,000 MB/s). **Workload:** cpu-only, 4,000 hosts, 10-second interval, 24-hour window (34.5M rows, 345.6M metrics). 1,000 queries per type, single client worker.

Query Type	Rate (q/s)	Median	Mean	Min	Max
single-groupby-5-1-1	1,314	0.77 ms	0.75 ms	0.23 ms	6.14 ms
single-groupby-1-1-1	1,254	0.68 ms	0.78 ms	0.20 ms	112.15 ms
single-groupby-1-8-1	858	1.10 ms	1.15 ms	0.85 ms	15.81 ms
single-groupby-5-8-1	667	1.31 ms	1.49 ms	1.06 ms	134.90 ms
single-groupby-1-1-12	625	1.42 ms	1.59 ms	0.98 ms	8.77 ms
single-groupby-5-1-12	574	1.58 ms	1.73 ms	1.15 ms	8.66 ms
lastpoint	507	2.38 ms	1.97 ms	1.38 ms	2.66 ms
cpu-max-all-1	486	1.86 ms	2.04 ms	0.95 ms	68.94 ms
high-cpu-1	213	4.02 ms	4.68 ms	2.26 ms	13.53 ms
cpu-max-all-8	154	5.96 ms	6.46 ms	4.39 ms	40.05 ms
groupby-orderby-limit	123	7.63 ms	8.12 ms	1.22 ms	24.38 ms
double-groupby-1	33	30.00 ms	30.37 ms	27.82 ms	145.05 ms
double-groupby-5	23	42.98 ms	43.36 ms	39.90 ms	339.10 ms
cpu-max-all-32-24	20	49.87 ms	48.97 ms	20.48 ms	264.10 ms
double-groupby-all	17	57.62 ms	57.62 ms	53.33 ms	143.94 ms
high-cpu-all	1	722.30 ms	711.85 ms	592.99 ms	839.36 ms

Narrow single-groupby queries return in sub-millisecond median latency at over 1,300 queries per second. Even the widest aggregations across all 4,000 hosts complete well under one second.

Feature comparison

Ecosystem and openness

Feature	QuestDB	kdb+
License	Apache 2.0 (open source)	Proprietary (commercial)
Independent benchmarks	Open, reproducible by anyone	Restricted (DeWitt clause)

Feature	QuestDB	kdb+
Storage format	Open (columnar + Parquet)	Proprietary binary format
Data portability	Parquet/Arrow readable by any compatible engine	Proprietary format, export needed
Data lake integration	Iceberg, Delta Lake, Hive	No native data lake integration

Infrastructure and operations

Feature	QuestDB	kdb+
Real-time / historical	Unified SQL across all tiers	Split RDB/HDB with separate processes
Security and auth	Built-in RBAC, TLS on by default	No auth or TLS out-of-the-box; DIY q gateways
Replication and DR	Built-in via shared object storage	DIY, requires custom q scripts
Query routing and sharding	Built-in	DIY, requires custom q gateways
Concurrency	Multi-threaded reads and writes; readers never block	Single-threaded core; only one writer per table
Storage and TCO	Low: separated storage/compute; single copy on object store	High: coupled storage/compute on local disks

Development and language

Feature	QuestDB	kdb+
Implementation	Zero-GC Java, C++, Rust	k / q
Query language	SQL (extended for time-series)	q (proprietary array language)
Time precision	Nanosecond timestamps	Nanosecond timestamps
Talent pool	SQL developers	Specialist q developers

AI and LLM readiness

Feature	QuestDB	kdb+
Open APIs and protocols	REST, PGWire, standard SQL; any LLM connects natively	LLMs fail 57% of q tasks on first try
Open data formats	Parquet/Arrow feeds directly into pandas, Polars, ML pipelines	Proprietary format; export required

Summary

kdb+ remains a capable engine with a large installed base. But its proprietary stack, niche language, and closed data formats carry trade-offs that compound over time. QuestDB ships fast: in 2025 alone, 16 releases delivered N-dimensional arrays, HORIZON JOIN, materialized views, and symbol auto-scaling. KDB-X, announced in late 2025, is now adopting open standards like Parquet, SQL, and PGWire, validating the direction QuestDB has built on from the start.

QuestDB vs. InfluxDB

InfluxDB is a time-series database developed by InfluxData. The open-source versions (v1 under MIT, v2 OSS under proprietary license) are written in Go. InfluxDB uses a measurement-based data model where each unique combination of tags creates a separate series with its own storage structure. Benchmarks cover InfluxDB v1.11, v2.7.12, and QuestDB 9.2.2.

Aspect	QuestDB	InfluxDB
License	Apache 2.0	v1: MIT, v2 OSS: Proprietary
Implementation	Java, C++	Go
Query language	Standard SQL	InfluxQL, Flux
Data model	Relational (tables + rows)	Measurement-based (series)
Ingestion protocols	ILP, PostgreSQL wire, HTTP	ILP, HTTP API
High cardinality	No performance impact	Performance degrades

Ingestion benchmark

QuestDB is 3.3x to 36x faster than InfluxDB, with the gap widening as cardinality increases.

Scale	InfluxDB v1.11	InfluxDB v2	QuestDB	vs v1	vs v2
100 hosts	1.23M rows/sec	727K rows/sec	4.02M rows/sec	3.3x	5.5x
1K hosts	1.17M rows/sec	667K rows/sec	7.48M rows/sec	6.4x	11.2x
4K hosts	787K rows/sec	514K rows/sec	8.39M rows/sec	10.7x	16.3x
100K hosts	491K rows/sec	402K rows/sec	11.36M rows/sec	23x	28x
1M hosts	~203K rows/sec	241K rows/sec	7.33M rows/sec	36x	30x

The gap widens because InfluxDB creates a separate TSM (Time-Structured Merge) tree for each unique series. At 100,000 hosts with 10 metrics each, that means 1,000,000 separate storage structures to maintain and compact. QuestDB stores all data in a single columnar table regardless of cardinality.

Query benchmark: Single-groupby

Query	InfluxDB v1.11	InfluxDB v2.7.12	QuestDB	Best
single-groupby-1-1-1	0.42 ms	0.73 ms	1.06 ms	InfluxDB v1
single-groupby-1-1-12	2.30 ms	3.37 ms	1.68 ms	QuestDB
single-groupby-1-8-1	1.00 ms	1.63 ms	1.39 ms	InfluxDB v1
single-groupby-5-1-1	1.09 ms	1.68 ms	0.99 ms	QuestDB
single-groupby-5-1-12	8.40 ms	12.24 ms	1.98 ms	QuestDB

Query	InfluxDB v1.11	InfluxDB v2.7.12	QuestDB	Best
single-groupby-5-8-1	3.23 ms	4.34 ms	1.54 ms	QuestDB

For simple aggregation on a single host (1-1-1), InfluxDB v1 is about 2.5x faster. As query complexity increases (more metrics, longer time ranges), QuestDB takes the lead: 4.2x faster on 5-metric/12-hour queries, 2.1x faster on 5-metric/8-host queries.

Query benchmark: Double-groupby

These queries aggregate across ALL 4,000 hosts, grouped by host and 1-hour intervals.

Query	InfluxDB v1.11	InfluxDB v2.7.12	QuestDB	vs v1	vs v2
double-groupby-1	853 ms	935 ms	40 ms	21x faster	23x faster
double-groupby-5	3,595 ms	3,875 ms	46 ms	78x faster	84x faster
double-groupby-all	6,967 ms	7,516 ms	58 ms	120x faster	130x faster

Query benchmark: Heavy analytical

Full table scan finding hosts with CPU utilization above threshold.

Query	InfluxDB v1.11	InfluxDB v2.7.12	QuestDB	vs v1	vs v2
high-cpu-all	16,045 ms	16,655 ms	994 ms	16x faster	17x faster

Data model differences

Aspect	QuestDB	InfluxDB
Data organization	Tables + rows	Measurements + series
Tag handling	SYMBOL columns (indexed strings)	Creates separate series per tagset
High cardinality	No impact (just more rows)	Performance degrades (more series = overhead)
Query language	Standard SQL	InfluxQL / Flux
JOINs	Full SQL JOIN support	Not supported
Schema	Schema-on-write or predefined	Schema-on-write

Query language: SQL vs. Flux

InfluxDB has gone through multiple query language changes: InfluxQL (SQL-like), then Flux (functional, now deprecated), and now SQL again in InfluxDB 3. This validates what QuestDB has maintained from the start.

Flux uses a functional pipeline syntax:

```
from(bucket: "metrics")
  |> range(start: -1h)
  |> filter(fn: (r) =>
       r._measurement == "cpu" and r.host == "server1")
  |> aggregateWindow(every: 1m, fn: mean)
```

The equivalent in QuestDB SQL:

```
SELECT timestamp, avg(usage)
FROM cpu
WHERE host = 'server1'
  AND timestamp > dateadd('h', -1, now())
SAMPLE BY 1m;
```

QuestDB time-series SQL extensions

Extension	Purpose	Example
SAMPLE BY	Time-based aggregation	`SELECT avg(price) FROM trades SAMPLE BY 1h`
LATEST ON	Last value per group	`SELECT * FROM trades LATEST ON timestamp PARTITION BY symbol`
ASOF JOIN	Time-aligned joins	Join trades with quotes at exact timestamps
WHERE IN	Time range filtering	Optimized partition pruning

Ecosystem and integrations

Integration	QuestDB	InfluxDB
Grafana	Native data source	Native data source
Telegraf	Via ILP	Native
PostgreSQL tools	Full compatibility (psql, any PG driver)	Not supported
Client libraries	Python, Java, Go, Node.js, Rust, C/C++, .NET	Python, Java, Go, Node.js, and more
Kafka	Official Kafka connector	Native Kafka consumer
Pandas/Polars	Native integration	Via client library

Performance summary

Workload	QuestDB	InfluxDB v2	Advantage
Ingestion (1M hosts)	7.33M rows/sec	241K rows/sec	30x faster
Ingestion (100K hosts)	11.36M rows/sec	402K rows/sec	28x faster
Double-groupby	40-58 ms	935 ms - 7.5s	23-130x faster
Heavy aggregations	994 ms	16.6s	17x faster

QuestDB vs. TimescaleDB

TimescaleDB is a time-series database built as a PostgreSQL extension. It inherits PostgreSQL's full SQL compatibility and ecosystem but also its row-based storage architecture. Benchmarks cover TimescaleDB 2.23.1 on PostgreSQL 17.6 and QuestDB 9.2.2.

Feature	QuestDB	TimescaleDB
Primary database model	Time Series DBMS	Time Series DBMS
Implementation language	Java (zero-GC), C++, Rust	C
SQL	Yes	Yes
APIs	ILP, HTTP, PGWire, JDBC	JDBC, ODBC, C lib

Ingestion benchmark

QuestDB is 6x to 13x faster than TimescaleDB.

Scale	QuestDB	TimescaleDB	Advantage
100 hosts	4.02M rows/sec	313K rows/sec	12.9x
1K hosts	7.48M rows/sec	1.24M rows/sec	6.0x
4K hosts	9.53M rows/sec	1.18M rows/sec	8.1x
100K hosts	11.36M rows/sec	1.03M rows/sec	11.0x
1M hosts	7.33M rows/sec	620K rows/sec	11.8x

TimescaleDB's performance is influenced by PostgreSQL's row-based architecture. Incoming writes go through PostgreSQL's B-Tree + heap file format before Hypercore's columnar conversion, adding latency compared to QuestDB's direct columnar writes.

Query benchmark: Single-groupby

Query	QuestDB (mean)	TimescaleDB (mean)	Advantage
1 metric, 1 host, 1h	1.07 ms	1.07 ms	Similar
1 metric, 1 host, 12h	1.73 ms	5.49 ms	3.2x faster
1 metric, 8 hosts, 1h	1.47 ms	3.62 ms	2.5x faster
5 metrics, 1 host, 1h	1.01 ms	1.15 ms	Similar
5 metrics, 1 host, 12h	1.84 ms	5.58 ms	3.0x faster
5 metrics, 8 hosts, 1h	1.47 ms	3.67 ms	2.5x faster

For simple point queries, both databases perform similarly. As complexity increases (longer time ranges, more hosts), QuestDB's advantage becomes 2.5x to 3.2x.

Query benchmark: Double-groupby

These queries aggregate across ALL 4,000 hosts, grouped by host and 1-hour intervals.

Query	QuestDB (mean)	TimescaleDB (mean)	Advantage
1 metric, all hosts	33.84 ms	547 ms	16x faster
5 metrics, all hosts	43.59 ms	789 ms	18x faster
10 metrics, all hosts	56.80 ms	1,135 ms	20x faster

Query benchmark: Heavy analytical

Query	QuestDB (mean)	TimescaleDB (mean)	Advantage
high-cpu-all	979 ms	1,082 ms	1.1x faster

For heavy analytical queries scanning large portions of the dataset, both databases perform comparably.

ClickBench: OLAP workload

ClickBench tests analytical query performance on web analytics data. Across most queries, QuestDB outperforms TimescaleDB by 10x to 650x on the same hardware:

Metric: Combined | Cold Run | Hot Run | Load Time | Storage Size

System & Machine	Relative time (lower is better). Different colors on the bar chart represent the same values shown at different scales (1x, 10x, 100x zoom)	
QuestDB (c6a.4xlarge):		×1.02
TimescaleDB (c6a.4xlarge):		×7.56
TimescaleDB (no columnstore) (c6a.4xlarge):		×216.56

Detailed Comparison

	QuestDB (c6a.4xlarge)	TimescaleDB (c6a.4xlarge)	TimescaleDB (no columnstore) (c6a.4xlarge)
Load time:	5403s (×4.27)	**1265s (×1.00)**	2633s (×2.08)
Data size:	67.84 GiB (×3.77)	**17.98 GiB (×1.00)**	67.08 GiB (×3.73)
Q0.	**0.000s (×1.00)**	0.036s (×4.60)	0.794s (×80.40)
Q1.	**0.062s (×1.00)**	0.164s (×2.42)	248.585s (×3452.71)
Q2.	**0.013s (×1.00)**	0.240s (×10.87)	248.325s (×10797.17)
Q3.	**0.039s (×1.00)**	0.202s (×4.33)	248.262s (×5066.78)
Q4.	**0.677s (×1.00)**	10.248s (×14.93)	263.995s (×384.29)
Q5.	**0.361s (×1.00)**	15.220s (×41.05)	270.091s (×728.04)
Q6.	**0.025s (×1.00)**	0.169s (×5.11)	248.046s (×7087.31)
Q7.	**0.062s (×1.00)**	0.183s (×2.68)	247.977s (×3444.26)
Q8.	**1.114s (×1.00)**	18.816s (×16.75)	277.788s (×247.15)
Q9.	**1.386s (×1.00)**	21.839s (×15.65)	281.095s (×201.36)
Q10.	**0.136s (×1.00)**	1.807s (×12.45)	248.555s (×1702.50)
Q11.	**0.136s (×1.00)**	2.361s (×16.24)	248.660s (×1703.22)
Q12.	**0.440s (×1.00)**	5.631s (×12.54)	256.911s (×570.94)
Q13.	**0.565s (×1.00)**	8.093s (×14.09)	258.080s (×448.85)
Q14.	**0.467s (×1.00)**	6.479s (×13.60)	258.061s (×541.03)
Q15.	**1.085s (×1.00)**	14.010s (×12.80)	273.497s (×249.78)
Q16.	**1.934s (×1.00)**	24.567s (×12.64)	314.388s (×161.73)
Q17.	**1.874s (×1.00)**	13.089s (×6.95)	301.501s (×160.04)
Q18.	**2.906s (×1.00)**	59.953s (×20.56)	361.127s (×123.85)
Q19.	**0.022s (×1.00)**	0.156s (×5.19)	247.374s (×7730.75)
Q20.	**0.422s (×1.00)**	1.420s (×3.31)	247.304s (×572.49)
Q21.	**0.424s (×1.00)**	1.429s (×3.32)	247.245s (×569.71)
Q22.	**0.403s (×1.00)**	1.589s (×3.87)	247.155s (×598.46)
Q23.	**0.018s (×1.00)**	0.079s (×3.18)	0.266s (×9.86)
Q24.	**0.001s (×1.00)**	0.043s (×4.82)	0.005s (×1.36)
Q25.	**0.111s (×1.00)**	0.409s (×3.46)	247.091s (×2042.16)
Q26.	**0.002s (×1.00)**	0.043s (×4.42)	0.005s (×1.25)
Q27.	**0.495s (×1.00)**	4.483s (×8.90)	247.028s (×489.18)
Q28.	**4.892s (×1.00)**	60.522s (×12.35)	260.902s (×53.23)
Q29.	**0.008s (×1.00)**	11.706s (×650.89)	246.867s (×13715.39)
Q30.	**0.366s (×1.00)**	7.236s (×19.27)	269.624s (×717.11)
Q31.	**0.424s (×1.00)**	14.866s (×34.28)	280.375s (×646.05)
Q32.	**4.456s (×1.00)**	139.289s (×31.19)	451.233s (×101.04)
Q33.	**2.405s (×1.00)**	27.704s (×11.48)	349.146s (×144.58)
Q34.	**2.352s (×1.00)**	29.356s (×12.43)	351.725s (×148.91)
Q35.	**0.872s (×1.00)**	10.723s (×12.17)	272.881s (×309.40)
Q36.	**0.085s (×1.00)**	0.266s (×2.91)	0.981s (×10.43)
Q37.	0.082s (×1.19)	**0.067s (×1.00)**	0.724s (×9.53)
Q38.	0.040s (×1.04)	**0.038s (×1.00)**	0.141s (×3.15)
Q39.	**0.103s (×1.00)**	0.559s (×5.04)	1.793s (×15.96)
Q40.	0.059s (×1.30)	**0.043s (×1.00)**	0.184s (×3.66)
Q41.	0.053s (×1.54)	**0.031s (×1.00)**	0.175s (×4.51)
Q42.	**0.011s (×1.00)**	0.041s (×2.43)	0.705s (×34.05)

Figure 3: ClickBench benchmark results: QuestDB vs TimescaleDB

ASOF JOIN comparison

TimescaleDB does not support joining tables based on the nearest timestamp. This join type is heavily used in financial markets. The workaround requires a verbose LEFT JOIN LATERAL:

QuestDB:

```
-- QuestDB provides ASOF JOIN. Since table definition includes
-- the designated timestamp column, no time condition is needed

SELECT t.*, n.*
FROM trades ASOF JOIN news ON (symbol);
```

PostgreSQL / TimescaleDB:

```
-- PostgreSQL and TimescaleDB offer no ASOF JOIN support,
-- but we can use a LEFT JOIN LATERAL to achieve similar results

SELECT t.*, n.*
FROM trades t LEFT JOIN LATERAL (
    SELECT *
    FROM news n1
    WHERE n1.symbol = t.symbol
    AND n1.timestamp <= t.timestamp
    ORDER BY n1.timestamp DESC
    LIMIT 1) n ON true
```

DuckDB also supports ASOF JOIN natively.

Architecture

TimescaleDB uses “hypertables” that partition PostgreSQL tables into “chunks” based on time intervals. Any unique index or primary key must include the time partitioning column. Incoming writes still go through PostgreSQL’s row-based format before Hypercore’s columnar conversion.

QuestDB is a purpose-built engine with no external dependencies. Its three-tier storage (WAL, columnar partitions, Parquet on object storage) writes directly to columnar format with SIMD-accelerated scans. QuestDB speaks the PostgreSQL wire protocol, so PostgreSQL client libraries work out of the box, providing PG ecosystem access without PG architecture constraints.

QuestDB vs. ClickHouse

ClickHouse is an OLAP engine that started at Yandex for e-commerce and ad tech analytics. It powers log monitoring, observability, and product analytics. The two databases serve different primary use cases: QuestDB is built for streaming ingestion and low-latency time-series queries; ClickHouse is optimized for batch-loaded OLAP workloads. Benchmarks cover ClickHouse 26.2.4.23 and QuestDB 9.3.3.

Aspect	QuestDB	ClickHouse
License	Apache 2.0	Apache 2.0
Implementation	Zero-GC Java, C++	C++
Query language	SQL with time-series extensions	SQL (ClickHouse dialect)
Data model	Relational (tables + rows)	Relational (tables + rows)
Ingestion protocols	ILP, PostgreSQL wire, HTTP	HTTP, Native protocol, Kafka, OpenTelemetry
Primary use case	Time series, capital markets, streaming	OLAP, observability (logs/traces/metrics)

Ingestion benchmark

QuestDB is 4.1x to 4.8x faster than ClickHouse with default configurations.

Scale	ClickHouse	QuestDB	Advantage
100 hosts	1.74M rows/sec	7.77M rows/sec	4.5x
1K hosts	1.74M rows/sec	7.77M rows/sec	4.5x
4K hosts	1.74M rows/sec	7.96M rows/sec	4.6x
100K hosts	1.78M rows/sec	8.59M rows/sec	4.8x
1M hosts	1.74M rows/sec	7.14M rows/sec	4.1x

ClickHouse holds steady throughput regardless of cardinality but at a lower baseline. ClickHouse can reach higher throughput with tuning (parallel threads, block sizes, async inserts), but that requires configuration work. QuestDB achieves these numbers with defaults.

Query benchmark: Single-groupby

Query	ClickHouse	QuestDB	Best
single-groupby-1-1-1	3.96 ms	1.00 ms	QuestDB 4x faster
single-groupby-1-1-12	3.98 ms	9.34 ms	ClickHouse 2.3x faster
single-groupby-1-8-1	4.63 ms	1.35 ms	QuestDB 3.4x faster
single-groupby-5-1-1	4.71 ms	0.97 ms	QuestDB 4.9x faster
single-groupby-5-1-12	4.85 ms	10.07 ms	ClickHouse 2.1x faster
single-groupby-5-8-1	6.67 ms	1.44 ms	QuestDB 4.6x faster

QuestDB is 3.4x to 4.9x faster on short-range point lookups. ClickHouse is 2.1x to 2.3x faster on 12-hour range queries, reflecting its strength in longer analytical scans.

Query benchmark: Double-groupby

Query	ClickHouse	QuestDB	Best
double-groupby-1	25.67 ms	31.55 ms	ClickHouse 1.2x faster
double-groupby-5	39.43 ms	43.15 ms	ClickHouse 1.1x faster
double-groupby-all	62.22 ms	58.55 ms	Tied

Query benchmark: Heavy and lastpoint

Query	ClickHouse	QuestDB	Best
high-cpu-1	19.54 ms	5.47 ms	QuestDB 3.6x faster
high-cpu-all	968.01 ms	724.65 ms	QuestDB 1.3x faster
lastpoint	41.79 ms	1.56 ms	QuestDB 27x faster
groupby-orderby-limit	10.77 ms	8.82 ms	QuestDB 1.2x faster

The lastpoint result (27x faster) reflects QuestDB's LATEST ON optimization, purpose-built for retrieving the most recent data point per key without scanning.

ClickBench: Web analytics workload

ClickBench tests analytical query performance on web analytics data (Yandex.Metrica hits). This workload plays to ClickHouse's strengths: wide tables and string-heavy analytics. Results below show hot run performance.

Metric: Combined | Cold Run | Hot Run | Load Time | Storage Size

System & Machine

QuestDB (c7a.metal-48xl):
ClickHouse (c8g.metal-48xl):
ClickHouse (c7a.metal-48xl):

Detailed Comparison

	QuestDB (c7a.metal-48xl)	ClickHouse (c8g.metal-48xl)	ClickHouse (c7a.metal-48xl)
Load time:	196s (×1.09)	**179s (×1.00)**	197s (×1.10)
Data size:	67.84 GiB (×4.77)	14.22 GiB (×1.00)	**14.22 GiB (×1.00)**
Q0.	**0.000s (×1.00)**	0.002s (×1.20)	0.002s (×1.20)
Q1.	0.019s (×1.38)	**0.011s (×1.00)**	**0.011s (×1.00)**
Q2.	**0.006s (×1.00)**	0.011s (×1.31)	0.012s (×1.38)
Q3.	**0.003s (×1.00)**	0.011s (×1.62)	0.012s (×1.69)
Q4.	**0.101s (×1.00)**	0.179s (×1.70)	0.335s (×3.11)
Q5.	**0.075s (×1.00)**	0.217s (×2.67)	0.337s (×4.08)
Q6.	**0.004s (×1.00)**	0.009s (×1.36)	0.010s (×1.43)
Q7.	**0.012s (×1.00)**	0.016s (×1.18)	0.016s (×1.18)
Q8.	0.318s (×1.60)	**0.195s (×1.00)**	0.274s (×1.39)
Q9.	0.361s (×1.73)	**0.205s (×1.00)**	0.282s (×1.36)
Q10.	0.126s (×1.51)	**0.080s (×1.00)**	0.112s (×1.36)
Q11.	0.102s (×1.45)	**0.067s (×1.00)**	0.078s (×1.14)
Q12.	0.137s (×1.75)	**0.074s (×1.00)**	0.085s (×1.13)
Q13.	0.156s (×1.54)	**0.098s (×1.00)**	0.125s (×1.25)
Q14.	0.073s (×1.11)	**0.065s (×1.00)**	0.077s (×1.16)
Q15.	0.074s (×1.50)	**0.046s (×1.00)**	0.052s (×1.11)
Q16.	**0.125s (×1.00)**	0.135s (×1.07)	0.144s (×1.14)
Q17.	0.124s (×1.07)	**0.115s (×1.00)**	0.117s (×1.02)
Q18.	**0.199s (×1.00)**	0.220s (×1.10)	0.256s (×1.27)
Q19.	0.009s (×1.58)	**0.002s (×1.00)**	0.003s (×1.08)
Q20.	0.054s (×1.10)	0.063s (×1.26)	**0.048s (×1.00)**
Q21.	0.045s (×1.34)	**0.031s (×1.00)**	0.038s (×1.17)
Q22.	**0.045s (×1.00)**	0.080s (×1.64)	0.082s (×1.67)
Q23.	**0.012s (×1.00)**	0.088s (×4.45)	0.079s (×4.05)
Q24.	**0.001s (×1.00)**	0.029s (×3.55)	0.027s (×3.36)
Q25.	**0.022s (×1.00)**	0.026s (×1.13)	0.024s (×1.06)
Q26.	**0.001s (×1.00)**	0.029s (×3.55)	0.027s (×3.36)
Q27.	0.065s (×1.92)	**0.029s (×1.00)**	**0.029s (×1.00)**
Q28.	**0.565s (×1.00)**	0.646s (×1.14)	0.710s (×1.25)
Q29.	**0.005s (×1.00)**	0.068s (×5.20)	0.046s (×3.73)
Q30.	0.059s (×1.19)	**0.048s (×1.00)**	0.062s (×1.24)
Q31.	**0.057s (×1.00)**	0.059s (×1.03)	0.069s (×1.18)
Q32.	**0.235s (×1.00)**	0.258s (×1.09)	0.360s (×1.51)
Q33.	**0.236s (×1.00)**	0.287s (×1.21)	0.320s (×1.34)
Q34.	**0.222s (×1.00)**	0.283s (×1.26)	0.330s (×1.47)
Q35.	0.065s (×1.74)	**0.033s (×1.00)**	0.042s (×1.21)
Q36.	0.042s (×1.08)	0.039s (×1.02)	**0.038s (×1.00)**
Q37.	0.034s (×1.38)	0.026s (×1.13)	**0.022s (×1.00)**
Q38.	0.035s (×1.55)	0.021s (×1.07)	**0.019s (×1.00)**
Q39.	0.099s (×1.45)	**0.065s (×1.00)**	0.086s (×1.28)
Q40.	0.031s (×1.58)	**0.016s (×1.00)**	0.017s (×1.04)
Q41.	0.036s (×2.00)	0.014s (×1.04)	**0.013s (×1.00)**
Q42.	**0.003s (×1.00)**	0.012s (×1.69)	0.011s (×1.62)

Figure 4: ClickBench benchmark results: QuestDB vs ClickHouse

Architecture: CREATE TABLE comparison

ClickHouse uses the MergeTree engine. Data is written to immutable parts (one compressed file per column + sparse index), which are continuously merged in the background:

```
CREATE TABLE trades (
    timestamp DateTime64(6),
    symbol String,
    exchange String,
    side String,
    price Float64,
    quantity Float64,
    trade_id UInt64
) ENGINE = MergeTree()
ORDER BY (symbol, timestamp);
```

QuestDB uses a three-tier architecture: WAL for durability and out-of-order handling, time-partitioned columnar files for query performance, and optional Parquet cold storage:

```
CREATE TABLE trades (
    timestamp TIMESTAMP,
    symbol SYMBOL,
    exchange SYMBOL,
    side SYMBOL,
    price DOUBLE,
    quantity DOUBLE,
    trade_id LONG
) TIMESTAMP(timestamp) PARTITION BY DAY;
```

ClickHouse's MergeTree requires background merges to consolidate parts, which compete with queries for resources. QuestDB's WAL writes directly to time-partitioned columnar storage, giving predictable write latency even under heavy load.

QuestDB time-series SQL extensions

Extension	Purpose	Example
SAMPLE BY	Time-based aggregation	`SELECT avg(price) FROM trades SAMPLE BY 1h`
LATEST ON	Last value per group	`SELECT * FROM trades LATEST ON timestamp PARTITION BY symbol`
ASOF JOIN	Time-aligned joins	Join trades with quotes at nearest timestamps
WINDOW JOIN	Rolling aggregation	Compute rolling stats within a time range around each row
HORIZON JOIN	Multi-offset markout analysis	Measure price evolution at multiple offsets after each trade
TICK syntax	Declarative timestamp filtering	`WHERE ts IN '2025-01-[01..31]#XNYS;6h30m'`

ClickHouse has no equivalent to SAMPLE BY, LATEST ON, WINDOW JOIN, HORIZON JOIN, or TICK syntax. Some can be approximated with verbose workarounds, but others like multi-offset markout analysis are not possible in ClickHouse.

QuestDB time-series query examples:

```
-- Hourly OHLCV bars for yesterday
SELECT timestamp, symbol,
    sum(quantity) AS volume,
    avg(price) AS avg_price
FROM trades
WHERE timestamp IN '$yesterday'
SAMPLE BY 1h;

-- Markout analysis: mid-price evolution
-- at 1s, 5s, 30s, 1m, 5m after each trade
SELECT t.symbol, h.offset,
    avg(q.mid - t.price) AS markout
FROM trades t
HORIZON JOIN quotes q ON (t.symbol = q.symbol)
    LIST (1s, 5s, 30s, 1m, 5m) AS h
WHERE t.timestamp IN '$today#XNYS'
GROUP BY t.symbol, h.offset;
```

Openness and data formats

QuestDB writes cold data to Apache Parquet on S3, Azure Blob, or GCS. Any tool that reads Parquet (DuckDB, Spark, Trino, pandas) can query it directly, with no QuestDB instance required. QuestDB exports metadata to Iceberg, Delta Lake, and Hive.

ClickHouse stores all data in its proprietary MergeTree format, including cold storage on S3. When ClickHouse tiers data to object storage, it remains in MergeTree format. To export it, you need a running ClickHouse instance. ClickHouse's cloud-native engine (SharedMergeTree) is closed-source and only available on ClickHouse Cloud.

Further reading

Read the full comparisons. Each comparison is covered in depth at questdb.com: the kdb+ comparison at questdb.com/compare/questdb-vs-kdb, and the InfluxDB, TimescaleDB, and ClickHouse comparisons in the QuestDB blog with full benchmark methodology and reproduction instructions.

Try QuestDB. QuestDB Open Source is available on Linux, macOS, Windows, Docker, and Kubernetes. Download and start a local instance in minutes for hands-on testing with your own data. A live demo instance at demo.questdb.com provides immediate access to over 2 billion rows of sample data with no installation required.

Evaluate Enterprise. Contact the QuestDB team at questdb.com/enterprise/contact for architecture reviews and guided deployment with Enterprise features including high availability, RBAC, TLS, and tiered storage.

Scan to get in touch